

Fundamentals

Driving the AI 🚗

How to direct AI tools effectively during the coding phase, and what to do when they fail.

You've oriented yourself and formed a plan. Now you're in the execution phase, where most of the interview happens. This is where you're actively prompting the AI, reviewing what it generates, and making steady progress through your plan. How you drive the AI during this phase is what interviewers spend the most time evaluating.

Prompting at the right granularity

The single biggest lever you have during execution is how much you give the AI in each prompt. Too vague and the AI makes all the decisions for you. Too specific and you're essentially dictating code line by line, which defeats the purpose of having an AI at all.

The sweet spot is giving the AI a clear *what* and *how* while letting it handle the syntax.

"Implement a trie that supports insert and prefix search, where each node stores a character and a boolean for end-of-word" is specific enough that the AI knows your approach, but open enough that it can write idiomatic code without you micromanaging every variable name.

Match your prompt specificity to the complexity of the task. For tricky logic, be precise about the approach, data structures, and constraints so the AI implements what you actually want. For boilerplate and repetitive code, a short underspecified prompt is fine. The AI knows how to write standard patterns and you're going to verify it anyway. If you're asking the AI to implement a complex state machine, be explicit about the states, transitions, and edge cases. If you're asking it to write five similar API endpoint handlers, a single short prompt is enough.

Also ask for concise output. AI tends to generate sprawling responses with lengthy comments and more abstractions than you need. In an interview, every extra line is something you have to review and potentially defend. A short "be concise" or "minimal comments" in your prompt goes a long way.

Your prompts should reference the actual codebase. Use the class names, method signatures, and data structures you learned during orientation. "Add a `searchByPrefix` method to the `Dictionary` class that uses the existing `TrieNode` structure and returns a `List<String>`" is a prompt that produces code fitting naturally into the existing architecture. A generic prompt

without this context will produce code that works in isolation but clashes with everything around it.



Using codebase-specific vocabulary in your prompts does double duty. It produces better AI output, and it signals to the interviewer that you actually understood the code during orientation. When you say "add a method to the `Dictionary` class" instead of "write a function that does autocomplete," the interviewer hears someone who knows the system they're working in.

When to use AI vs. when to write it yourself

Not everything in the interview should go through the AI. If you can write something in 30 seconds, just write it. A small bug fix, a one-line change, renaming a variable. Prompting the AI for these things takes longer than doing it yourself, and it creates an impression that you can't code without assistance.

On the other hand, anything that would take you five minutes but the AI can produce in ten seconds is a clear candidate for prompting. Standard patterns, data structure implementations you know are correct but tedious to type, test case generation, and boilerplate setup code all fall into this category.



There's a nuance here that's specific to interviews. In your day-to-day work, prompting the AI for everything makes sense because there's no downside to getting a second perspective. In an interview, the calculus is different. Companies are evaluating whether you're an engineer who uses AI as a tool, or just a prompt relay. The interview has to surface your thinking: what you know, how you reason, what decisions you make. If the AI is making all the calls and you're just accepting the output, the interviewer learns nothing about you.

The general rule is to use AI for volume and yourself for judgment. Let the AI handle the parts where speed matters and correctness is easy to verify. Handle the parts where the approach itself is the hard part, where choosing the wrong data structure or missing an edge case would cost you significant time later.

Keep your code running

Run the code after every meaningful AI generation. Don't stack multiple changes without compiling in between — each generation might be individually correct, but they can conflict in ways that produce a cascade of errors when you finally try to run everything. Fix breakages before moving on. Never layer new code on top of broken code.

This also gives you natural checkpoints to narrate to the interviewer: "That compiles and passes the existing tests, so now I'm moving on to the optimization phase." We cover testing strategy and incremental verification in depth in [verification and testing](#).

When AI gives you bad output

AI will occasionally give you output that doesn't work or doesn't match what you asked for. How you handle this matters more than whether it happens, because the interviewer knows AI isn't perfect.

Before talking about tactics, it helps to know the common failure patterns so you can spot them quickly.

The most common is **training data bias**. Ask for a solution to a graph traversal problem and you'll often get the textbook leetcode solution — complete and correct, but not necessarily the right fit for the specific constraints of your problem. Maybe the codebase already has utilities that make a simpler solution possible, or maybe the problem has a twist that makes a different approach more efficient. Recognizing when the AI gave you a technically correct but strategically wrong answer is critical.

The second pattern is **verbosity**. AI generates more code than you need, more abstractions than you asked for, and more comments than are helpful. You end up with 150 lines where 60 would do, and now you have to understand and defend all of it. Get in the habit of asking yourself what can be simplified without losing functionality.

The third is **design shortcuts**. AI loves creating base classes, deep inheritance hierarchies, and over-engineered abstractions. It'll introduce a `BaseProcessor` superclass for two classes that share almost nothing, or wrap everything in a strategy pattern when a simple if-else would be clearer. These choices might not break anything, but they signal to the interviewer that you accepted a lazy design without thinking critically about it.

The fourth is **correctness shortcuts**. AI will sometimes delete failing tests instead of fixing the underlying issue, use `any` to bypass type errors, add `try/catch` blocks that swallow exceptions, or hardcode values to make specific test cases pass. These are red flags to an interviewer. They suggest you either didn't notice or didn't care. Always scan generated code for these patterns before accepting it.

With those patterns in mind, here's how to respond. The worst move is prompting the same thing again and hoping for a different result. If the AI didn't understand what you wanted the first time, repeating yourself won't help. Rephrase your prompt with more context, be more specific about the approach you want, or break the request into smaller pieces. Often the issue is that your prompt was ambiguous and the AI interpreted it differently than you intended.



In open-ended interviews, commit frequently. If the AI starts going off the rails and you've got several bad generations layered on top of each other, rolling back to the last working state is almost always faster than trying to manually undo the damage. A clean rollback point beats ten minutes of debugging AI-introduced chaos.



Watch out for the "almost right" trap. When AI output is 90% correct, there's a strong temptation to manually patch the remaining 10% line by line. This is usually slower than reprompting with a clearer request, and it produces messy code that's half AI-generated and half hand-edited. If the output missed the mark, ask again properly rather than surgically fixing what you got.

In open-ended formats you technically have the option to switch models, but in practice this is rarely worth it. Reprompting with more specificity or just writing it yourself is almost always faster than switching context to a different model mid-interview.

Sometimes the fastest path is to just write it yourself. If you've spent two minutes trying to get the AI to produce the right output and you could write the ten lines of code in one minute, switch to manual. This actually looks good to the interviewer. It shows you're pragmatic about your tools and don't have a dependency on AI to function.



A good rule of thumb is one reprompt. If the first attempt misses, rephrase once with more specificity. If that also misses, either switch models or write it yourself. Spending more than two minutes fighting with AI on a single generation is almost never worth it.

The "nerfed AI" problem



We're not certain the interview AI is intentionally nerfed. What's more likely is that the tools in structured interview environments lack the system prompts and context that tools like Claude Code or Cursor have baked in, which makes them feel substantially less capable. The practical effect is the same: don't expect the interview AI to behave like your daily driver.

Multiple candidates at [Meta](#) have reported the interview AI feeling noticeably worse than what they practiced with. At Uber, candidates describe it as giving nudges rather than solutions. The AI might not point out bugs directly, give cryptic or incomplete responses, or refuse to help with certain types of requests.

If you encounter this, adjust your expectations and lean more on your own understanding. Use the AI for what it can do, generating boilerplate, explaining unfamiliar syntax, scaffolding basic structures, and handle the harder thinking yourself. The interview is partly testing whether you can work effectively with imperfect tools, which is honestly what real engineering looks like most of the time anyway.

The candidates who struggle here are the ones who practiced exclusively with state-of-the-art models and never developed a workflow for when the AI can only get them part of the way. If you followed the advice in [how to prepare](#) about practicing with limited AI, this won't catch you off guard.